2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS) | 979-8-3503-3071-7/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICPADS60453.2023.00018

SR2C: A Structurally Redundant Short Reads Collapser for Optimizing DNA Data Compression

Hui Sun[1, †], Huidong Ma[1, †], Yingfeng Zheng[1, †], Haonan Xie[2], Xiaofei Wang[1], Xiaoguang Liu[1,*], Gang Wang[1,*]
[1]College of C.S., Nankai-Orange D.T. Joint Lab, Nankai-Baidu Joint Lab, ICIC, TMCC, SysNet, DISSec, GTIISC, Nankai University, Tianjin 300350, China
[2]School of Electrical Engineering, Guangxi University, Nanning 530004, China
[†] These authors contribute equally to this work
[*]Corresponding authors: {liuxguang, wgzwp}@nbjl.nankai.edu.cn

***Abstract*—The current redundant sequence deduplication algorithms cannot remove structural repetitive DNA short reads such as mirror, reverse, paired, and complementary palindromes in high-throughput genomics sequencing data. Moreover, these methods also cannot construct indexes to recover the original sequences, thus failing to meet the requirements of lossless compression for downstream applications. To address these problems, we propose a data structure called Cycle-Hash-Linkage (CHL) and present a CPU parallelism optimization algorithm named SR2C (Structurally Redundant Short Reads Collapser) based on CHL to improve the compression ratio of DNA sequencing data. Experimental results on actual data from the NCBI database demonstrate that SR2C achieves an average residual sequence percentage improvement of 2.556% compared to the state-of-the-art redundant sequence deduplication algorithm, Minirmd. Furthermore, SR2C cascaded optimization improves the average compression ratios of compression algorithms Pigz, PBzip2, XZ, and 7Z by 92.345%, 78.999%, 10.132%, and 7.434%, respectively. By leveraging multi-core CPU parallel computation, SR2C effectively reduces time consumption, which achieves 2-5X deduplication and recovers acceleration.**

The same name Linux toolkit is freely available at https://github.com/fahaihi/SR2C.

***Index Terms*—parallel algorithm, redundancy deduplication, DNA sequencing data compression, structurally redundant reads, data compression, rolling-hash algorithm**

I. INTRODUCTION

With the rapid development of sequencing technologies like BGISEQ [1] and IlluminaNGS [2], High Throughput Genomics Sequencing Data (HTGSD) has experienced significant growth, following the trajectory of Moore's. For instance, by December 2023, the China National GeneBank (CNGB) already archived 12,390 terabytes (TB) of HTGSD [3]. Typically, HGSTD comprises descriptive information, DNA short reads, and quality score data. Among HTGSD, DNA short reads are characterized by simple symbolic composition and high data redundancy, with approximate repeats accounting for 32% of DNA short reads [4]. To mitigate the potential PCR amplification bias during library construction and reduce the computational resources required for downstream applications, it is essential to remove near-repeated sequencing reads from large-scale biological data. This ensures faster and more efficient read alignment, whole-genome association analysis, and sequencing reads data compression [4]–[6].

Existing redundancy sequencing short reads deduplication methods are classified into alignment-based and de novo-based approaches [5]. Alignment-based methods, such as Markdup [6] and Gencore [7], map reads to the reference genome and remove redundant information based on the relative position of reads in the reference genome. However, the alignment-based methods have some limitations in usage, such as the time required for read alignment to the reference genome and the proper reference genome selection.

Methods based on de novo utilize the similarities among short reads to decrease redundancy, and they do not need any previous information about a reference genome, which makes them very promising for different usage scenes. De novo-based methods include exact-based and approximate-based approaches. In the first category, exact repeat deletion techniques reduce redundant information by comparing the hash fingerprints of different reads, and such representative approaches include NGSReadsTreatment [8], BioSeqZip [9], and Nubeam-dedup [10]. However, exact-based methods do not consider mismatches between sequencing reads data, making them unable to handle near-repeated reads with mismatches. The second category, which consists of approximate-based methods considering mismatches, requires similarity alignment for each read. Regrettably, genomics sequencing reads data typically reach up to millions. When applying the edit distance algorithm [11] to eliminate mismatched repeat reads from m short reads with a length of n, the complexity can reach up to $O(m^2 \times n^2)$. Some methods remove approximate duplicate reads by clustering them into various groups, reducing the time required to calculate pairwise similarities for large datasets. This way, instead of comparing every file to each other, they only compare files within the same group. For example, Fulcrum [12], ParDRe [13], and MarDRe [14] group similar reads together based on string prefixes. Minirmd [5] clusters similar reads using the minimizer hash algorithm,

2690-5965/23/$31.00 ©2023 IEEE
DOI 10.1109/ICPADS60453.2023.00018

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

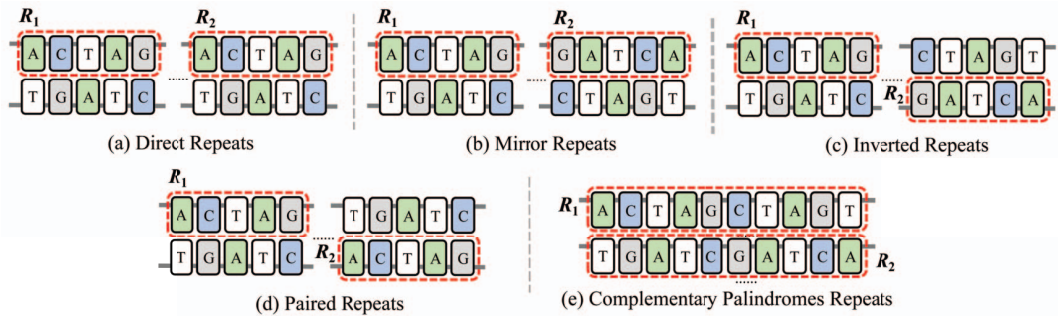


Fig. 1: Example of 5 structural repeat short reads in genome sequencing data

while CD-HIT-DUP [15] utilizes a greedy incremental clustering algorithm to group similar reads.

In our observation, those above-mentioned short reads redundancy deduplication techniques effectively remove approximate reads with low mismatches. However, they cannot handle structurally repeated reads in genomic data. (In Section II, we will explain the sequencing short reads that are considered to be "*structurally redundant*"). Furthermore, approximate-based strategies cannot construct indexes for recovering the original reads for lossless sequencing data compression applications. To address those problems, we design a Cycle-Hash-Linkage data structure for removing structurally redundant short reads in genomics sequencing data. We also present a multi-core CPU accelerate short reads collapser called SR2C that employs CHL for optimizing genomic sequencing data lossless compression.

The main innovations and contributions of this paper can be summarized as follows:

- By integrating rolling hash, hamming distance, linkage list, and hash buckets, we propose a CHL data structure that efficiently mines and indexes redundant strings in DNA short read collection, especially those structurally repetitive sequences.
- Constructing upon CHL, we implemented SR2C, a data collapser specifically tailored for optimizing genomics sequencing short reads data compression. The proposed SR2C achieves lossless compression of short reads via the presented data streams encoding algorithm DeDupCHL and decoding algorithm RecoCHL.
- Our algorithm implementation employs multi-core CPUs to speed up the computation of SR2C through a data cycle division parallel strategy. This parallel mechanism reduces the time SR2C runs, especially during decoding, while maintaining high redundancy substitution.

The rest of this paper is organized as follows: Section II describes the proposed CHL data structure and the sequencing data lossless compression optimization method SR2C in detail. Section III gives the experimental environment, data description, and experimental analysis. Section IV provides the conclusion and future outlook.

II. METHODS

Let $\Sigma = \{\text{A, C, G, T, N}\}$ denotes the alphabet of composed DNA sequencing short reads, $R = \{R_1, R_2, ..., R_{m-1}\}$ represents a collection of string reads, n denotes the average length, and m is the number of short reads. In this section, we first provide the implementation points of the CHL data structure and then deliver the parallel redundant short reads encoding algorithm and decoding algorithm of the SR2C optimization method in detail.

A. Cycle-Hash-Linkage Data Structure

The high throughput genomics sequencing reads exhibit a simplistic symbolic arrangement and considerable data redundancy [4], [16]. Along with Direct Repeats (DRs), there are several other types of repeats found in DNA reads, including Mirror Repeats (MRs), Inverted Repeats (IRs), Paired Repeats (PRs), and Complementary Palindromes Repeats (CPRs).

Fig. 1 provides an example of these five types of repeat short reads in genomics data. As shown in Fig. 1, we can see that the repeated read R_2 in Fig. 1(b-d) can be replaced by similarity reads R_1 through string transformation, looping, and shifting operations, which are different from Fig. 1(a). Therefore, our intuition is to cross-reference m strings greedily to optimize reads data lossless compression by utilizing "*redundant structural information*" between these strings.

Let $Hash(R)$ denotes the hash fingerprint [17] of read R. If R_1 and R_2 are direct repeat reads of each other, then we have $Hash(R_1) = Hash(R_2)$. At the same time, we arranged $R \overset{so}{\Longrightarrow} R_0$, $R \overset{re}{\Longrightarrow} R_1$, $R \overset{co}{\Longrightarrow} R_2$, and $R \overset{cr}{\Longrightarrow} R_3$ to denote the source, reverse, complementary, and complementary reverse operations of DNA read R, respectively, where base A and T, C and G, N and N are complementary. For example, let the source read R =ACGTA, then we will get reads R_0=ACGTA, R_1=ATGCA, R_2=TGCAT, and R_3=TACGT. To extract more structurally repeated short reads, we can utilize cycle shift transformation on the above four basic operations in a greedy manner. Let $R_1^e \overset{k}{\Longrightarrow} R_2^{e,k}$ represents R_1^e transfers to $R_2^{e,k}$ after k cyclic right shift operations, $R_1^e \overset{k}{\Longleftarrow} R_2^{e,k}$ means that the $R_2^{e,k}$ becomes R_1^e after k cyclic left shift operations, where $e = 0, 1, 2, 3$, and

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

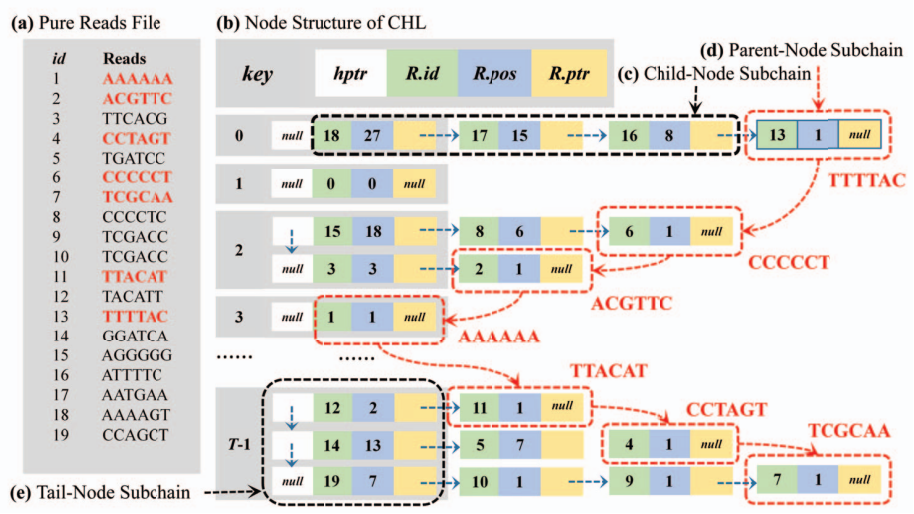


Fig. 2: Example of de-duplicating 19 sequencing reads by using CHL data structure.

$k = 0, 1, 2, ..., n-1$. A specific example, for instance, if R_1 is equal to ACGTN and R_2 is equal to GCANT, R_1 can be transformed into R_2 through one reverse operation and two proper right loop operations, resulting in R_1 becoming R_2 (i.e., ACGTN$\overset{re}{\Longrightarrow}$NTGCA$\overset{k=2}{\Longrightarrow}$GCANT). At the same time, we can also repair R_2 to R_1 via inverse transformation, that is, ACGTN$\overset{re}{\Longleftarrow}$NTGCA$\overset{k=2}{\Longleftarrow}$GCANT.

Therefore, based on the uncomplicated but practical string structure transformation above, we can produce the same hash fingerprints for the five types of repeated short reads shown in Fig. 1 to establish cross-references.

- DRS: $Hash(R_1) = Hash(R_2) \overset{so}{\Longrightarrow} R_2^0 \overset{k=0}{\Longrightarrow} R_2^{0,0}$
- MRS: $Hash(R_1) = Hash(R_2) \overset{re}{\Longrightarrow} R_2^0 \overset{k=0}{\Longrightarrow} R_2^{0,0}$
- IRS: $Hash(R_1) = Hash(R_2) \overset{re}{\Longrightarrow} R_2^0 \overset{k=0}{\Longrightarrow} R_2^{0,0}$
- PRS: $Hash(R_1) = Hash(R_2) \overset{so}{\Longrightarrow} R_2^0 \overset{k=0}{\Longrightarrow} R_2^{0,0}$
- PRS: $Hash(R_1) = Hash(R_2) \overset{co}{\Longrightarrow} R_2^0 \overset{k=0}{\Longrightarrow} R_2^{0,0}$

By exploiting the reverse, complementary, and complementary reverse operations on the m source short reads, redundant sequences that are "*structural repeats*" of each other can be discovered by comparing the hash fingerprints between the m reads. Furthermore, we can accurately identify shift repeats caused by high sequencing coverage by performing the appropriate cyclic shift operation on four repeated read states: source, reverse, complementary, and complementary reverse sequencing reads. However, shifting the short reads cyclically and performing the hash calculation can be time-consuming, particularly when dealing with millions of reads. If $4 \times n$ shift operations are performed for each read, the total hash calculation time complexity for SR2C will achieve $O(4 \times m \times n^2)$. To avoid the redundant hash computation, we also introduce Rolling Hash [18], [19]to optimize the hash computation time complexity to $O(m \times n)$. In addition, to avoid the hash collision problem, we also introduced the Hamming Distance algorithm [20] for collision correction. Therefore, our proposed CHL can be understood as an efficient short reads redundancy removal data structure, which combines efficient rolling hash, linkage list, hamming distance, and hash bucket. To better comprehend the notion of duplication deletion of structurally redundant short reads in this paper, Fig. 2 illustrates an instance of de-duplicating 19 sequencing short reads by using the presented CHL data structure.

In Fig. 2, Fig. 2(a) contains 19 DNA sequencing short reads, where reads with id 16, 17, and 18 generate similarity substitutions with id 13; reads with id 8 and 15 perform similarity substitutions by read with id 6; reads with id 2 and 3, reads with id 11 and 12, and reads with id 5,14, and 4 are structurally redundant reads to each other; reads with id 9, 10, and 19 can generate structural substitutions by read with id 7. Fig. 2(b) symbolizes the node structure of the CHL, where id denotes the relative file index, pos is the relative position index, ptr is the node pointer, the key represents the mapping position of the hash fingerprint in the hash table generated by the rolling hash function, $hptr$ is the pointer structure for resolving hash conflicts, and T denotes the hash bucket size. Fig. 2(c) pictures the Child-Node Subchain (CNS) generated by the CHL data structure, which records the redundant reads' relative file and location index information. Fig. 2(d) represents the Parent-Node Subchain (PNS) generated by the CHL data structure, which records the parent reads' relative file and location index information. CHL also introduces the Tail-Node Subchain (TNS) structure shown in Fig. 2(e) and uses the Hamming Distance algorithm [20] for conflicting reads checks to avoid generating hash conflicts. In Fig. 2, the relative position index $R.pos$ consists of the source, reverse, complementary, complementary reverse, and cycle right shift operation of read R. Where $R.pos = n \times e + k + 1$, and

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: DeDupCHL

Input: DNA short reads collection $R = \{R_1, R_2, ..., R_m - 1\}$. Parameters m, n, and Pr.
Output: Encoding data streams $fids$, $freads$, $poss$, ids, $counts$, and $infos$.

1 Initialize the hash table H via the parameter m ; /* The hash bucket size T takes $1.3 \times m$ */
2 **for** $p = 0$ **to** $Pr - 1$ **para do**
3 **for** $i = 0$ **to** $m - 1$ **para do**
4 **if** $p == \lceil \frac{i}{Pr} \rceil$ /* Pr CPU cores use shared memory model for parallel computing */ **then**
5 **for** $e = 0$ **to** 3 **do**
6 **for** $k = 0$ **to** $n - 1$ **do**
7 Calculate the hash fingerprint $hvalue \leftarrow RHash(R_i^{e,k})$ of $R_i^{e,k}$ by rolling hash function ;
8 **for** $j = 0$ **to** $|H[hvalue]|$ **do**
9 Perform hamming check on the DNA read and $R_i^{e,k}$ recorded in the corresponding subchain PNS in the j-th sub-chain CNS to obtain the sequence hamming check value $hanvalue$;
10 **if** $hanvalue == 0$ **then**
11 Construct a child node $CN \leftarrow \{i+1, e \times n + k + 1, null\}$, and add the CN to the j-th child node's subchain ; /* No hash conflict */
12 **goto** (line 3) ; /* R_i is a redundant read. Update hash table over */
13 **if** $H\left[RHash(R_i^{0,0})\right] == null$ **then**
14 Create a TNS $H\left[RHash(R_i^{0,0})\right].TNS$, and update the $FN \leftarrow \{null, i+1, 0, null\}$ to the TNS ;
15 **else**
16 Create a new CNS and update node $FN \leftarrow \{null, i+1, 1, null\}$ to the CNS;
17 Add the CNS constructed by line 16 to the tail node subchain $H\left[RHash(R_i^{0,0})\right].TNS$;
18 Iterate over the hash table H, and store the data streams $fids$, $freads$, $poss$, ids, $counts$, and $infos$;

$e = 0, 1, 2, 3$. For instance, the read R_{18} can produce similarity substitution by R_{13} through 21 cycle shift operations, that is, $Hash(R_{13}) = Hash(R_{18} \overset{cr}{\Longrightarrow} R_{18}^{3} \overset{k=3}{\Longrightarrow} R_{13})$. Thus, the corresponding file index $R.id$ of R_{18} is 13, and the relative position index $R.pos$ takes $21(6 \times 3 + 2 + 1)$.

B. Parallel Redundant Short Reads Encoding

In the redundant short reads encoding stage, we aim to utilize the CHL data structure, as mentioned earlier, to encode redundant short reads and decrease the compressed data size. Specifically, the storage space of one sequencing read, with length n, will be up to $8 \times n$ bits. If CHL can replace redundant read, only 32 bits of pos and id information must be reserved on a 32-bit operating system. Then the read replaced by similarity only occupies $8/n$ of the original storage space.

The CHL data structure retains the sequence reads' position and redundant substitution information for lossless compression of the de-duplicated genomics sequencing files. To recover the source reads in the lossless compression scenario of reads data, SR2C introduces the following six data streams to encode redundant reads.

- ***fids***: a collection records relative file index id of the parent sequencing reads in the Parent-Node Subchain.
- ***freads***: a collection of sequencing reads recorded in the Parent-Node Subchain.
- ***poss***: a set of relative position indexes pos of the subreads in the Child-Node Subchain.
- ***ids***: a set of sub-reads relative position index id recorded in the Child-Node Subchain.
- ***counts***: a collection records the lengths of each Child-Node Subchain.
- ***infos***: a statistical information set related to reads after deduplication in CHL, including the reads length n, the total number of reads m, and the size of $|fids|$.

Let Pr represents the SR2C used CPU cores, $CN = \{id, pos, ptr\}$ and $FN = \{hptr, id, pos, ptr\}$ denote CHL nodes that record reads information, where FN is used in the subchain TNS, CN is used in the subchain FNS and CNS. Algorithm 1. formally describes the algorithm DeDupCHL based on CHL to deduplicate and encode redundant reads using a data cycle division parallel strategy [21], [22].

In parallel Algorithm 1, the time consumption of step 1 is $O(m)$; the worst time of steps 5-17 to find and update the redundant sequence of a single read structure is $O(4 \times n)$; the time of steps 2-17 to parallel iterate the m reads to construct the CHL table is $O(\frac{4 \times m \times n}{Pr})$; the total time overhead

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

of step 18 to iterate the hash table and save the data streams is $O(m)$. Thus, the time complexity of the parallel algorithm DeDupCHL is $O(\frac{m \times n}{Pr})$. The memory space required for the reads collection R during the operation of Algorithm 1 is $O(m \times n)$; the space required to construct the circle hash table is $O(4 \times m)$. Therefore, the space complexity of the parallel algorithm DeDupCHL is $O(m \times n)$.

C. Parallel Redundant Short Reads Decoding

The parallel algorithm DeDupCHL iterates the structurally redundant subreads of the reads collection $R = \{R_1, R_2, ..., R_{m-1}\}$ by constructing a CHL data structure to find the equivalent parent sequence of the structurally repeated subreads. To lossless recover the source reads, the optimization method SR2C performs a left cyclic shift operation on the parent sequences based on the six data streams saved in the algorithm DeDupCHL. Algorithm 2 formally describes the presented multi-core CPU parallel algorithm RecoCHL for decoding redundant sequencing reads by using the DeDupCHL algorithm generated data streams $fids$, $freads$, $poss$, ids, $counts$, and $infos$.

Algorithm 2: RecoCHL

Input: Data streams $fids$, $freads$, $poss$, ids, $counts$ and $infos$, and parameter Pr.
Output: Reads collection $R = \{R_1, R_2, ..., R_{m-1}\}$.

1 Load parameters m, n, $|fids|$ using stream $infos$;
2 Initialize a two-dimensional array $R[m][n] = \{\phi\}$;
3 **for** $p = 0$ **to** $Pr - 1$ **para do**
4 **for** $i = 0$ **to** $|fids| - 1$ **para do**
5 **for** $j = \sum_{t=0}^{i-1} counts[t]$ **to** $\sum_{t=0}^{i} counts[t]$ **do**
6 $Fread \leftarrow R[fids[i]]$;
7 $k \leftarrow (poss[j] - 1)\%n$;
8 **if** $poss[j] == 1$ and $p == \lceil \frac{i}{Pr} \rceil$ **then**
9 $R[ids[j]] \leftarrow Fread$;
10 **else if** $1 < poss[j] < n + 1$ and $p == \lceil \frac{i}{Pr} \rceil$ **then**
11 $R[ids[j]] \leftarrow (Fread^{so,k} \overset{k}{\Longleftarrow} Fread)$;
12 **else if** $n < poss[j] < 2 \times n + 1$ and $p == \lceil \frac{i}{Pr} \rceil$ **then**
13 $R[ids[j]] \leftarrow (Fread^{re,k} \overset{k}{\Longleftarrow} Fread)$;
14 **else if** $2 \times n < poss[j] < 3 \times n + 1$ and $p == \lceil \frac{i}{Pr} \rceil$ **then**
15 $R[ids[j]] \leftarrow (Fread^{co,k} \overset{k}{\Longleftarrow} Fread)$;
16 **else**
17 $R[ids[j]] \leftarrow (Fread^{cr,k} \overset{k}{\Longleftarrow} Fread)$;
18 Iterate over the $R[m][n]$ to lossless recover the set of sequencing reads $R = \{R_1, R_2, ..., R_{m-1}\}$;

Let $t = \frac{\left(\sum_{i=0}^{|fids|-1} counts[i]\right)}{|fids|}$. During the run of Algorithm 2, the time consumption of steps 1-2 is $O(m \times n)$; the time of the inner for-loop of steps 5-17 is $O(t \times n)$. The time of the outer for-loop of steps 3-17 is $O(\frac{|fids| \times t \times n}{Pr})$. The parameters $|fids|$ and t are much smaller than the parameter m. Thus, the time complexity of algorithm RecoCHL is $O(\frac{m \times n}{Pr})$. The space required for the array $R[m][n]$ is $O(m \times n)$; the maximum memory space required for $fids$, $poss$, ids, $counts$, and $infos$ is $O(m)$. Therefore, the space complexity of the parallel algorithm RecoCHL is $O(m \times n)$.

III. RESULTS AND DISCUSSION

This section first introduces the experimental environment and benchmark datasets, followed by experimental performance and analysis of the SR2C CPU parallel algorithm.

A. Experimental Environment and Datasets

Our proposed method SR2C was tested at Sugon 7000A super parallel computer system of the National Supercomputing Center Nanning Branch Center (https://hpc.gxu.edu.cn/index.htm). The experimental node configuration is 2*Intel Xeon Gold 6230 CPU (40 cores), 192 GB DDR4 memory, and 8*900 GB disk storage space. The node runs CentOS7.4 with a 64-bit version of the operating system. The SR2C optimization method is programmed using C++11 and OpenMP.

The experiment utilizes eight open-source datasets covering different species and sequencing platforms from the NCBI (https://www.ncbi.nlm.nih.gov) database [23] for performance evaluation. The total size of these data is 25,607,473 KB, containing 105,016,192 sequencing short reads. A detailed description of the experimental datasets is shown in TABLE I. We have packaged SR2C into a same-name Linux toolkit and given detailed installation and configuration tutorials, datasets acquisition methods, and supplementary performance results at https://github.com/fahaihi/SR2C.

TABLE I: Details of experimental benchmark datasets

Datasets	Platform	R.Number	F.Size (KB)
C.arietinum	HiSeq 2000	36160656	11078656
Human	DNBSEQ-T7	5193329	930355
M.fascicularis-1	BGISEQ-500	1981367	3561225
M.fascicularis-2	BGISEQ-500	19811367	3605669
Mouse.tumor-1	MGISEQ-2000RS	2446512	689917
Mouse.tumor-2	MGISEQ-2000RS	2446512	685472
S.fontinalis-1	Ion Torrent Proton	2055479	532203
S.fontinalis-2	Ion Torrent Proton	17090970	4523976
Total	—	105016192	25607473

B. deduplication Performance

To verify the structurally redundant reads deduplication effect of the proposed SR2C, the experiments tested the percentage of remaining reads obtained by running the SR2C cascading with state-of-the-art mismatched repeated reads removal algorithm Minirmd [5]. The percentage of remaining reads is defined as the ratio of de-duplicated remaining reads to the total number of sequencing reads. The lower percentage of remaining reads indicates the better performance of the

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Percentage of remaining sequencing short reads obtained by running Minirmd cascading with SR2C

Datasets	**Without SR2C**				**With SR2C**			
	$NMis = 0$	$NMis = 1$	$NMis = 2$	$NMis = 3$	$NMis = 0$	$NMis = 1$	$NMis = 2$	$NMis = 3$
C.arietinum	76.531	69.474	64.010	59.771	**70.625**	**65.576**	**60.879**	**57.035**
Human	21.846	19.659	18.918	18.519	**21.221**	**19.58**	**18.871**	**18.476**
M.fascicularis-1	11.764	6.123	4.897	4.050	**11.719**	**6.080**	**4.860**	**4.013**
M.fascicularis-2	11.45	6.365	5.151	4.490	**11.415**	**6.328**	**5.115**	**4.454**
Mouse.tumor-1	17.008	8.365	4.358	2.699	**16.839**	**8.359**	**4.354**	**2.696**
Mouse.tumor-2	18.772	9.464	4.973	3.088	**18.624**	**9.454**	**4.966**	**3.082**
S.fontinalis-1	33.079	30.660	28.705	27.053	**33.069**	**30.653**	**28.700**	**27.048**
S.fontinalis-2	19.848	17.495	15.967	14.720	**19.569**	**17.491**	**15.964**	**14.717**
Average	26.288	20.951	18.372	16.799	**25.385**	**20.442**	**17.964**	**16.440**
Improvement (%)	—	—	—	—	**3.436%**	**2.428%**	**2.225%**	**2.134%**

TABLE III: The number of remaining reads obtained by running different cascaded orders of SR2C and Minirmd

Datasets	**Before (SR2C + Minirmd)**				**After (Minirmd + SR2C)**			
	$NMis = 0$	$NMis = 1$	$NMis = 2$	$NMis = 3$	$NMis = 0$	$NMis = 1$	$NMis = 2$	$NMis = 3$
C.arietinum	**25538394**	24296798	22446940	20950331	**25538394**	**23712805**	**22014323**	**20624364**
Human	**1102078**	1031124	988391	964280	**1102078**	**1017296**	**980033**	**959526**
M.fascicularis-1	**2321698**	1477161	1054721	862405	**2321698**	**1205610**	**962876**	**794989**
M.fascicularis-2	**2261396**	1484747	1100134	935340	**2261396**	**1253703**	**1013267**	**882339**
Mouse.tumor-1	**411960**	240016	126955	76225	**411960**	**204506**	**106516**	**65947**
Mouse.tumor-2	**455642**	268821	142467	85862	**455642**	**231292**	**121500**	**75404**
S.fontinalis-1	**679731**	635914	592869	557902	**679731**	**630072**	**589932**	**555965**
S.fontinalis-2	**3344524**	3021908	2743668	2524700	**3344524**	**2989442**	**2728446**	**2515288**
Average	**36115423**	32456489	29196145	26957045	**36115423**	**31244726**	**28516893**	**26473822**
Improvement (%)	—	—	—	—	0	**3.733%**	**2.327%**	**1.793%**

algorithm [5], [8], [9]. Table II displays the experimental results on eight datasets when the mistake matching parameter [5] $NMis$ is taken as 0 to 3, respectively.

The experimental results in TABLE II displayed that in the Without SR2C computation mode, the Minirmd algorithm retained 26.288%, 20.951%, 18.372%, and 16.799% remaining sequencing reads percentages when the parameter $NMis$ was taken as 0 to 3, respectively. With SR2C mode, the remaining percentages were 25.385%, 20.442%, 17.964%, and 16.440%, respectively. Compared with Without SR2C mode, the algorithm Minirmd cascaded with SR2C to improve the percentage by 3.436%, 2.428%, 2.225%, and 2.134%, respectively. The overall experimental results show that benefiting from the replacement impacts of the CHL data structure, SR2C shows a good deduplication effect.

In order to find out how the using order of SR2C and Minirmd [5] affects the remaining sequencing reads, we also conducted experiments with various cascaded orders. The results are presented in TABLE III. According to the results presented in TABLE III, it was found that the After mode, which involved using the SR2C algorithm after the cascade, was more effective in eliminating duplicate reads than the Before mode. When $NMis = 0$, the Before and After modes were taken with the same results at the remaining sequencing reads. The reason was that when $NMis$ took the value of 0, the Minirmd algorithm only performed one hash fingerprinting operation on the short reads collection, so the cascading order using the SR2C algorithm accomplished not affect the results. When $NMis$ took the 1, 2, and 3 values, the percentage of remaining reads boosted in After mode compared to Before mode is 3.773%, 2.327%, and 1.793%, respectively. This was because the Minirmd needed to remove the approximate reads according to the parameter $NMis$. When the Before mode was used for execution, the SR2C first removed the structurally redundant reads in the dataset, which made the Minirmd algorithm unable to find the duplicate reads similar to the removed structurally redundant reads and reduced the number of redundant reads identified by the algorithm Minirmd.

C. Compression Performance

The SR2C algorithm stands out from other deduplication methods like Minirmd [5], Fulcrum [12], CD-HIT-DUP [15], and ParDRe [13] in that it can restore the source reads without any loss. This is possible because SR2C retains the original relative position index information and uses it to decode the data stream. To evaluate the effectiveness of SR2C parallel optimized sequencing reads lossless compression, the experiments tested the compression effect of SR2C cascading with four state-of-the-art general compression algorithms Pigz (V2.7) [24], PBzip2 (V1.1.13) [25], XZ (V5.2.3) [26], and 7Z (V22.01) [27]. To ensure fairness, we have configured the comparison algorithm to use the best compression mode and set the number of parallel cores to $Pr = 8$. TABLE IV displays the compression ratios achieved by cascading SR2C with four universal algorithms on eight datasets. The compression ratio refers to the size ratio of sequencing reads before and after compression. A higher compression ratio suggests better algorithm performance [28].

The results exhibited in TABLE IV, the average compression ratios of Pigz, PBzip2, XZ, and 7Z algorithms on all eight datasets were 5.164, 6.912, 13.645, and 13.806,

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

TABLE IV: Compression ratios of SR2C cascading with compressors Pigz, PBzip2, XZ, and 7Z

Datasets	**Compression Without SR2C**				**Compression With SR2C**			
	Pigz	PBzip2	XZ	Pigz	Pigz	PBzip2	XZ	Pigz
C.arietinum	3.820	4.082	**6.144**	**6.174**	**4.336**	**4.609**	5.925	5.939
Human	**8.384**	**10.630**	**10.636**	**10.725**	7.864	8.027	10.628	10.610
M.fascicularis-1	3.380	3.549	10.332	10.743	**8.460**	**9.433**	**11.698**	**11.270**
M.fascicularis-2	3.380	3.549	10.253	10.652	**8.524**	**9.510**	**11.660**	**11.241**
Mouse.tumor-1	6.994	12.101	24.142	24.237	**14.567**	**18.227**	**28.816**	**28.603**
Mouse.tumor-2	6.938	11.860	23.594	23.687	**13.908**	**17.454**	**27.535**	**27.331**
S.fontinalis-1	4.225	4.786	**12.015**	**12.003**	**7.743**	**8.579**	11.979	11.955
S.fontinalis-2	4.187	4.740	12.041	12.226	**9.923**	**11.035**	**14.687**	**14.783**
Average	5.164	6.912	13.645	13.806	**9.416**	**10.859**	**15.366**	**15.216**
Improvement (%)	—	—	—	—	**92.345%**	**78.999%**	**10.132%**	**7.434%**

respectively, when SR2C mode was not used. However, when SR2C mode was utilized, the average compression ratios of these algorithms improved significantly to 9.416, 10.859, 15.366, and 15.2165, respectively. Among these algorithms, Pigz and PBzip2 compressors showed the most remarkable improvement in compression ratio, i.e., 150.273% and 165.762%, respectively, on the M.fascicularis-1 dataset when SR2C cascading was employed with four compression algorithms. On the other hand, algorithms XZ and 7Z showed the most significant improvement in compression ratio, i.e., 150.273% and 165.762%, respectively, on the S.fontinalis-2 dataset. Moreover, algorithms XZ and 7Z exhibited a maximum improvement in compression ratio of 21.975% and 20.914%, respectively, on the S.fontinalis-2 dataset. Overall, the algorithms Pigz, PBzip2, XZ, and 7Z showed an average improvement in compression ratio of 92.345%, 78.99%, 10.132%, and 7.434%, respectively, on all eight datasets.

SR2C encodes the redundant sequencing short reads as data streams. Since the encoded data streams decrease the size of the reads to be compressed compared with the original files, the data size of the encoded data streams is much smaller than the replaced structurally redundant reads. When using SR2C to optimize universal compression, cascaded compressors can acquire a higher magnitude of compression ratio improvement to reduce the storage space of short reads.

D. Parallel Performance

The SR2C utilizes multi-core CPUs to speed up computationally intensive steps in parallel, including CHL hash bucket update, Rolling-Hash calculation, and lossless restoration of the original sequencing short reads collection. To evaluate the effectiveness of parallel acceleration, we conducted experiments to measure the number of remaining reads (NRR), deduplication time (DT), deduplication memory (DM), lossless recovery time (RT), and lossless recovery memory (RM) achieved by SR2C on the three different file scale datasets C.arietinum, Human, and M.fascicularis-1 datasets with varying numbers of CPU cores. The sizes of the three files are 10.565GB, 0.887GB, and 3.396GB, respectively, and contain 36,160,656, 5,193,329, and 1,981,367 sequencing short reads, respectively. The experimental results are shown in V.

As shown in TABLE V, three sets of results were collected from the Human dataset regarding the number of remaining reads (NRR). Similarly, two different sets of results were obtained from the M.fascicularis-1 dataset. This variation occurred due to the algorithm SR2C using a hash function, which resulted in an unavoidable introduction of error mapping issues. However, SR2C relies on hamming distance for short reads verification, allowing the algorithm to accurately and successfully lossless recover source data, even when errors or mapping issues are present. Regarding deduplication time (DT), parallel acceleration accomplished not deliver a significant advantage because of the strong coupling of circular hash chains with data. Limited by the competitive access of multi-core CPUs to hash table critical area resources, SR2C reached the peak value of DT when the number of cores was taken 2-3. However, it was necessary to remark that SR2C took a significant advantage in parallel recovery time (RT). The optimization method SR2C reached the peak RT when the number of cores was set to 10 on the three datasets. As the number of cores increases, the memory consumption (DM & RM) for deduplication and lossless recovery increases slightly. This is because additional memory space is required for multi-core parallelism.

IV. CONCLUSION AND FUTURE WORK

In this paper, we design a Cycle-Hash-Linkage data structure and propose a sequencing short reads compression optimization method SR2C based on CHL. Experimental results show that the average remaining sequencing reads percentage of SR2C cascading with Minirmd is improved by 2.556% in removing redundant reads. In terms of compression ratio optimization, the overall average compression ratio of SR2C cascading with compression algorithms Pigz, PBzip2, XZ, and 7Z is improved by 47.228%, which effectively reduces the storage size of sequencing data and reduces the construction expenditure of basic storage facilities. Furthermore, the SR2C CPU parallelization algorithm exhibited a significant acceleration effect on the benchmark datasets, particularly in parallelizing the restoration of the initial data.

With the emergence of third-generation sequencing technologies like Oxford Nanopore, the sequencing reads are distinguished by their lengthy size of about 150 Kbp and a high error rate of 15-20% [21]. This paper introduces the SR2C, which significantly improves the compression ratio of

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.

TABLE V: Parallel performance of SR2C on C.arietinum, Human and M.fascicularis-1 datasets

Datasets	**Metrics**	$Pr = 1$	$Pr = 2$	$Pr = 4$	$Pr = 6$	$Pr = 8$	$Pr = 10$
C.arietinum	NRR	**25538394**	**25538394**	**25538394**	**25538394**	**25538394**	**25538394**
	DT (HH:MM:SS)	01:07:36	01:02:59	**00:54:53**	01:19:39	02:16:29	02:26:34
	DM (KB)	**11172032**	11172484	11172552	11172188	11172452	11172452
	RT (HH:MM:SS)	27:20:28	21:03:24	12:53:29	09:09:46	07:24:41	**06:04:04**
	RM (KB)	5251564	5250972	5250732	**5250224**	5250796	5250560
Human	NRR	**1102078**	1102079	1102079	1102080	1102079	1102079
	DT (HH:MM:SS)	00:01:44	**00:01:16**	00:02:14	00:02:47	00:03:13	00:03:46
	DM (KB)	1018132	1016524	1015808	**1015792**	1018492	1020916
	RT (HH:MM:SS)	00:01:47	00:01:20	00:00:46	00:00:39	00:00:30	**00:00:26**
	RM (KB)	**581272**	581900	583064	582636	582484	584692
M.fascicularis-1	NRR	**2321698**	**2321698**	**2321698**	**2321698**	**2321698**	2321699
	DT (HH:MM:SS)	00:04:21	**00:03:56**	00:05:23	00:05:56	00:06:29	00:06:43
	DM (KB)	3586864	3586336	**3586288**	3586380	3586408	3586340
	RT (HH:MM:SS)	00:07:37	00:05:51	00:03:38	00:02:35	00:02:10	**00:01:46**
	RM (KB)	2172064	**2172016**	2172032	2172248	2172072	2172200

sequencing short reads. To address the challenges posed by large-scale, long sequencing with high error rates reads, future research based on this work will focus on three areas. Firstly, concerning the unnecessary data attributes in long reads and researching the approximate sequence deletion algorithm for these long reads is essential. Additionally, it is a promising research area to delve further into utilizing the redundant information between files and creating a specialized parallel compression algorithm for large-scale genomics sequencing data. Lastly, heterogeneous parallel cluster systems can potentially optimize the compression of large-scale sequencing data, especially with the emergence of various hardware computing devices such as GPUs and TPUs. Exploring how to separate sequential operations and develop a reads collection lossless compression parallel optimization algorithm for heterogeneous acceleration devices is also worth researching.

ACKNOWLEDGMENTS

This work is partly supported by NSF of China (62141412, 62272253, 62272252), Fundamental Research Funds for the Central Universities. This work was also supported by the high performance computing center of Guangxi University.

REFERENCES

[1] Li, FayWei, and Alex Harkess. "A guide to sequence your favorite plant genomes". Applications in Plant Sciences, 2018, vol.6(3), pp.e1030.
[2] Hu T, Chitnis N, Monos D, et al. "Next-generation sequencing technologies: An overview". Human Immunology, 2021, vol.82(11), pp.801-811.
[3] Guo X, Chen F, Gao F, et al. "CNSA: a data repository for archiving omics data". Database, 2020, pp.baaa055.
[4] Ji Z, Zhou J R, Jiang L. "Overview of DNA sequence data compression techniques". Acta Electronica Sinica, 2010, vol.38(5), pp.1113-1121.
[5] Liu Y, Zhang X, Zou Q, et al. "Minirmd: accurate and fast duplicate removal tool for short reads via multiple minimizers". Bioinformatics, 2021, vol.37(11), pp.1604-1606.
[6] Li H, Handsaker B, Wysoker A, et al. "The Sequence alignment map (SAM) format and SAMtools". Bioinformatics, 2009, vol.25(16), pp.2078-2079.
[7] Chen S, Zhou Y, Chen Y, et al. "Gencore: an efficient tool to generate consensus reads for error suppressing and duplicate removing of NGS data". BMC Bioinformatics, 2019, vol.20(23), pp.1-8.
[8] Gaia A S C, de Sá P H C G, de Oliveira M S, et al. "NGSReadsTreatment–a Cuckoo Filter-based tool for removing duplicate reads in NGS data. "Scientific Reports, 2019, vol.9(1), pp.11681.
[9] Urgese G, Parisi E, Scicolone O, et al. "BioSeqZip: a collapser of NGS redundant reads for the optimization of sequence analysis". Bioinformatics, 2020, vol.36(9), pp.2705-2711.
[10] Dai H, Guan Y. "Nubeam-dedup: a fast and RAM-efficient tool to deduplicate sequencing reads without mapping". Bioinformatics, 2020, vol.36(10), pp.3254-3256.
[11] Marzal A, Vidal E. "Computation of normalized edit distance and applications". IEEE transactions on pattern analysis and machine intelligence, 1993, vol.15(9), pp.926-932.
[12] Burriesci M S, Lehnert E M, Pringle J R. "Fulcrum: condensing redundant reads from high-throughput sequencing studies". Bioinformatics, 2012, vol.28(10), pp.1324-1327.
[13] González-Domínguez J, Schmidt B. "ParDRe: faster parallel duplicated reads removal tool for sequencing studies". Bioinformatics, 2016, vol.32(10), pp.1562-1564.
[14] Expósito R, Veiga J, González-Domínguez J, et al. "MarDRe: efficient MapReduce-based removal of duplicate DNA reads in the cloud". Bioinformatics, 2017, vol.33(17), pp.2762-2764.
[15] Huang Y, Niu B, Gao Y, et al. "CD-HIT Suite: a web server for clustering and comparing biological sequences". Bioinformatics, 2010, vol.26(5), pp.680-682.
[16] Ji Z, Zhou J, Zhu Z, Wu Q. "Bioinformatics Features Based DNA Sequence Data Compression Algorithm". Acta Electronica Sinica, 2011, vol.39(5), pp.991-995.
[17] Bernal-Romero J C, Ramirez-Cortes J M, Rangel-Magdaleno J, et al. "A review on protection and cancelable techniques in biometric systems". IEEE Access, 2023, vol.11, pp.8531-8568.
[18] Jiang H. "Rolling hashing based on carry less multiplication and its implementation in LZ4 data compression". Doctor: University of Science and Technology of China, 2021.
[19] Pokharana, Anchal, and Urvashi Garg. "A Review on diverse algorithms used in the context of Plagiarism Detection." 2023 International Conference on Advancement in Computation and Computer Technologies (InCACCT). IEEE, 2023.
[20] Garzón E, Yavits L, Teman A, et al. "Approximate content-addressable memories: a review". Chips, 2023, vol.2(2), pp.70-82.
[21] Zhong C, Sun H. "Parallel algorithm for sensitive sequence recognition from long-read genome data with high error rate". Journal on Communications, 2023, vol.44(02), pp.160-171.
[22] Cheng, John, Max Grossman, and Ty McKercher. Professional CUDA c programming. John Wiley and Sons, 2014.
[23] Geer LY, Marchler-Bauer A, Geer RC, et al. "The NCBI biosystems database". Nucleic acids research, 2010, col.38(s1), pp.492-496.
[24] Pigz. "Pigz Official Website". 2023, https://zlib.net/pigz/.
[25] PBzip2. "Official Website". 2023, https://linux.die.net/man/1/pbzip2
[26] XZ. "XZ Official Website". 2023, https://tukaani.org/xz/.
[27] 7-Zip. "7-Zip Official Website". 2023, https://www.7-zip.org/.
[28] Shaohui Xie, Xiaotian He, Shan He. et al. "CURC: a CUDA-based reference-free read compressor". Bioinformatics, 2023, vol.38(12), pp.3294–3296.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:21:27 UTC from IEEE Xplore. Restrictions apply.